تنافسي البرامجيات

جامعة واسط - كلية علوم الحاسوب وتكنولوجيا المعلومات

https://t.me/cs_mscX

ملخص جابتر 3

المصفوفات، القوائم المتصلة والاستدعاء الذاتي

Arrays, Linked Lists, and Recursion

من المصدر:

Data Structures & Algorithms in C++, 2nd, ed Michael T. Goodrich

إعداد: قناة تنافسي البرامجيات

المصفوفات (Array)

تُعد المصفوفات من هياكل البيانات الملموسة والأساسية التي يتم فيها **الوصول إلى المدخلات باستخدام فهارس عددية صحيحة** .يستعرض هذا القسم تطبيقات المصفوفات والمفاهيم النظرية المتعلقة باستخدامها في تنظيم البيانات.

1.تطبيقات المصفوفات وخصائصها

1. **التخزين الأساسي :**تُستخدم المصفوفات بشكل شائع لتخزين مجموعة من الكائنات، مثل سجلات نقاط الألعاب العالية، أو سجلات المرضى في المستشفى، أو أسماء اللاعبين في فريق.
2. **هيكلة المدخلات :**عادةً ما يتضمن الكائن الممثل لمدخل واحد (مثل سجل نقاط الألعاب) مكونًا عدديًا يمثل النتيجة وآخر يمثل اسم الشخص.
3. **تنظيم البيانات :**عند تصميم هيكل لتخزين النتائج العالية، يتم تخزين أعلى النقاط في مصفوفة (تسمى في المصادر (`entries`، مع الاحتفاظ بمتغير يحدد **الحد الأقصى لعدد الإدخالات** المسموح بها(`maxEntries`) ، ومتغير آخر لتتبع **العدد الفعلي** للإدخالات الموجودة حاليًا.(`numEntries`)
4. **الترتيب :**يتم اختيار ترتيب مدخلات المصفوفة بناءً على قيمها، وعادةً ما يتم ترتيب سجلات النقاط من **الأعلى إلى الأدنى.**

2.عمليات الإضافة (Insertion) في المصفوفات المرتبة

تتمثل عملية إضافة إدخال جديد (`e`) في مجموعة الدرجات العالية في ضمان بقاء المصفوفة مرتبة مع الالتزام بالحد الأقصى لحجمها.

- **التحقق من الامتلاء والإهمال :**إذا كانت المصفوفة ممتلئة بالفعل، يتم تجاهل الإدخال الجديد إذا كانت نتيجته أقل أو تساوي نتيجة آخر إدخال (وهو الأقل قيمة).

- **الإزاحة :(Shifting)** لإضافة إدخال جديد، يجب **إزاحة جميع الإدخالات** ذات النقاط الأصغر من نقطة الإدخال الجديد إلى اليمين بمقدار موضع واحد لإفساح المجال للإدخال الجديد.
- **آلية الإزاحة :**لتجنب الكتابة فوق المدخلات الموجودة، تبدأ عملية الإزاحة من **الطرف الأيمن** للمصفوفة وتعمل باتجاه اليسار.
- **أداء الإضافة :**يمكن أن تكون عملية الإضافة بطيئة نسبيًا إذا تطلبت إزاحة عدد كبير من الإدخالات الموجودة.

3.عملية الإزالة (Removal) من المصفوفات

تتضمن عملية إزالة كائن عند فهرس معين (`i`) تحديث المصفوفة لملء الفجوة الناتجة عن حذف الكائن.

- **الإزاحة لليسار :**تتم إزالة العنصر عبر تحريك جميع الإدخالات التي تقع في فهارس أعلى من الفهرس `i` **موضعًا واحدًا إلى اليسار** لملء مكان العنصر المحذوف.
- **تسلسل العملية :**تبدأ عملية الإزاحة من الموضع الذي يلي موضع الحذف مباشرة وتستمر من اليسار إلى اليمين؛ هذا الترتيب ضروري لتجنب الكتابة فوق الإدخالات الموجودة.
- **القيمة المرتجعة :**يجب حفظ قيمة الإدخال الذي يتم إزالته في متغير مؤقت قبل حذفه، ليتم إرجاعها كناتج للعملية.

4.فرز المصفوفات(Sorting an Array)

الفرز هو عملية إعادة ترتيب عناصر المصفوفة التي قد تكون مرتبة بشكل عشوائي، بحيث تصبح بترتيب غير تنازلي (تصاعدي).

- **فرز بالإدراج :(Insertion-Sort)** هي خوارزمية فرز بسيطة.

- **آلية عمل الفرز بالإدراج :**تبدأ الخوارزمية بعنصر واحد (يُعتبر مرتبًا)، ثم تتناول كل عنصر تالٍ تكراريًا وتقوم بإدراجه في **الجزء الذي تم فرزه بالفعل** من المصفوفة (المصفوفة الفرعية التي تقع إلى يساره)، ويتم ذلك عبر إزاحته يسارًا حتى يصل إلى موضعه الصحيح.
- **الحالة الأفضل :**إذا كانت المصفوفة مرتبة بالفعل، فإن الحلقة الداخلية في الخوارزمية تجري مقارنة واحدة فقط لكل عنصر، وتستنتج أنه لا يلزم إجراء أي مبادلة.
- **الحالة الأسوأ :**تحدث عندما تكون المصفوفة الأولية مرتبة ترتيبًا تنازليًا.

5.المصفوفات ثنائية الأبعاد(Two-Dimensional Arrays)

تُستخدم المصفوفات ثنائية الأبعاد لتمثيل البيانات المخزنة في **صفوف وأعمدة**.

- **الاستخدامات :**تُعد طريقة طبيعية لتمثيل الكائنات في فضاء ثنائي الأبعاد، وتُستخدم في الألعاب الموضعية (Positional Games) مثل لوحات الألعاب.
- **الفهرسة :**يشير الفهرس الأول عادةً إلى رقم **الصف**، بينما يشير الفهرس الثاني إلى رقم **العمود**.
- **التعريف الهيكلي :**تُعرف المصفوفة ثنائية الأبعاد أحيانًا باسم **المصفوفة (Matrix)**. في لغة++C ، يتم تعريف المصفوفة ثنائية الأبعاد على أنها **مصفوفة من المصفوفات**، حيث يكون كل عنصر من عناصر المصفوفة الرئيسية هو صف (مصفوفة أخرى).

التخصيص الديناميكي للمصفوفات ثنائية الأبعاد(Dynamic Allocation)

عندما لا تكون أبعاد المصفوفة معروفة مسبقًا، يجب تخصيصها ديناميكيًا.

- **آلية التخصيص :**بما أن المصفوفة هي مصفوفة من الصفوف، وكل صف هو مصفوفة ديناميكية، يجب أولاً تخصيص مصفوفة من **مؤشرات الصفوف**، ثم يتم تخصيص كل صف على حدة (مصفوفة من الأعداد الصحيحة مثلاً).
- **آلية إلغاء التخصيص :**تتطلب عملية إلغاء التخصيص عكس الخطوات؛ يجب أولاً حذف (إلغاء تخصيص) كل صف على حدة، ثم حذف مصفوفة مؤشرات الصفوف.
- **بديل :STL** تُسهل فئة المتجهات القياسية (STL vector class) عملية تخصيص المصفوفات الديناميكية عن طريق تطبيق المصفوفة كـ **متجه من المتجهات**، مما يجعلها تتولى تلقائيًا عملية حذف الأعضاء وإلغاء التخصيص.

مثال تطبيقي: الألعاب الموضعية

تستخدم المصفوفات ثنائية الأبعاد في الألعاب الموضعية، مثل لعبة إكس-أو-Tic) Tac-Toe).

- **تمثيل اللوحة :**يتم استخدام مصفوفة ثنائية الأبعاد (لوحة (boardللاحتفاظ بلوحة اللعبة، حيث تخزن الخلايا قيمًا تشير إلى ما إذا كانت الخلية فارغة أو تحتوي على علامة اللاعب X أو.O
- **التحقق من الفوز :**يمكن استخدام ترميز عددي) مثل 0 لخلية فارغة، 1 للاعبX ، و -1 للاعب .(O هذا الترميز يتيح اختبارًا بسيطًا للفوز: إذا كان مجموع قيم صف أو عمود أو قطر يساوي 3 أو -3، فهذا يشير إلى فوز اللاعب X أوO ، على التوالي.

القوائم المتصلة (Linked Lists)

تُعد القوائم المتصلة من هياكل البيانات الديناميكية التي توفر بديلاً مرنًا للمصفوفات، حيث تتكون من مجموعة من العُقد المرتبطة معًا لتشكيل ترتيب خطي. يستعرض هذا القسم المفاهيم النظرية الأساسية والخصائص البنيوية للقوائم المتصلة بأنواعها المختلفة.

1.المفهوم الأساسي والبنية العامة

1.1التعريف الهيكلي

- **القائمة المتصلة** :هي مجموعة من العُقد (Nodes) تشكل معًا تسلسلاً خطيًا، حيث تُمثل كل عُقدة وحدة مستقلة في الذاكرة.
- **العُقدة :(Node)** الوحدة الأساسية في القائمة المتصلة، تتكون من جزأين رئيسيين:
  - **البيانات :(Data/Element)** القيمة المخزنة في العُقدة.
  - **المؤشر :(Pointer/Link)** رابط يشير إلى موقع العُقدة التالية أو السابقة في الذاكرة.

1.2المكونات الأساسية للقائمة

- **الرأس :(Head)** مؤشر يشير إلى أول عُقدة في القائمة، ويُعتبر نقطة الدخول للوصول إلى القائمة بأكملها.
- **الذيل :(Tail)** مؤشر يشير إلى آخر عُقدة في القائمة، ويسهل عمليات الإضافة في النهاية.
- **القيمة الفارغة :(NULL/nullptr)** تُستخدم للدلالة على نهاية القائمة أو غياب الارتباط.

1.3الخصائص البنيوية العامة

- **عدم التجاور في الذاكرة** :على عكس المصفوفات، لا تُخزن عناصر القائمة المتصلة في مواقع متتالية بالذاكرة، بل تكون متفرقة ومرتبطة عبر المؤشرات.

- **الحجم الديناميكي** :لا يُحدد حجم القائمة مسبقًا، ويمكن تغييره بإضافة أو حذف العُقد حسب الحاجة دون إعادة تخصيص الذاكرة.
- **الاجتياز التسلسلي :(Sequential Access)** للوصول إلى عُقدة معينة، يجب البدء من الرأس والانتقال تسلسليًا عبر العُقد باتباع المؤشرات.

1.4تخطي الروابط(Link Hopping/Traversal)

- **المفهوم** :عملية الانتقال من عُقدة إلى أخرى عن طريق اتباع مؤشر العُقدة التالية.(next)
- **الآلية** :يبدأ الاجتياز من الرأس، ويستمر باتباع المؤشرات حتى الوصول إلى العُقدة المطلوبة أو نهاية القائمة.
- **التعقيد الزمني** :يتطلب الوصول إلى عُقدة في موضع معين اجتياز جميع العُقد السابقة لها.

.2القوائم المتصلة الأحادية(Singly Linked Lists)

2.1التركيب النظري

- **التسمية** :تُسمى "أحادية" لأن كل عُقدة تحتوي على مؤشر واحد فقط يشير إلى الاتجاه الأمامي.(next)
- **بنية العُقدة** :تتكون من:
  - حقل البيانات(data/element)
  - مؤشر واحد (next pointer) يشير إلى العُقدة التالية
- **نهاية القائمة** :يحتوي مؤشر العُقدة الأخيرة (الذيل) على قيمة فارغة (NULL)للدلالة على نهاية القائمة.

2.2المزايا النظرية

- **المرونة في الحجم** :لا يتطلب تحديد الحجم مسبقًا، ويمكن النمو والانكماش ديناميكيًا.
- **كفاءة الإدراج والحذف في البداية** :عمليات الإضافة والحذف من الرأس تتم بتعقيد زمني ثابت.

- **عدم الحاجة للإزاحة** :لا تتطلب عمليات الإدراج والحذف تحريك العناصر الأخرى كما في المصفوفات.
- **استخدام الذاكرة الفعلي** :تستخدم الذاكرة فقط للعناصر الموجودة بالفعل.

2.3القيود النظرية

- **اتجاه واحد للاجتياز** :يمكن الاجتياز في اتجاه واحد فقط (من الأمام للخلف)، ولا يمكن الرجوع للعقد السابقة مباشرة.
- **عدم الوصول العشوائي** :لا يمكن الوصول مباشرة إلى عُقدة بموضع معين دون اجتياز العُقد السابقة.
- **صعوبة الحذف من النهاية** :لحذف العُقدة الأخيرة، يجب اجتياز القائمة بأكملها للوصول إلى العُقدة قبل الأخيرة.
- **ذاكرة إضافية للمؤشرات** :كل عُقدة تحتاج مساحة إضافية لتخزين المؤشر.

2.4العمليات الأساسية

2.4.1الإضافة في المقدمة(addFront/insertFront)

- **المفهوم** :إدراج عُقدة جديدة لتصبح الرأس الجديد للقائمة.
- **الخطوات النظرية:**
  1. إنشاء عُقدة جديدة تحتوي على العنصر المطلوب
  2. جعل مؤشر العُقدة الجديدة يشير إلى الرأس الحالي
  3. تحديث مؤشر الرأس ليشير إلى العُقدة الجديدة
- **الحالة الخاصة** :إذا كانت القائمة فارغة، تصبح العُقدة الجديدة هي الرأس والذيل معًا.
- **التعقيد الزمني** :ثابت، لا يعتمد على حجم القائمة.

2.4.2الإضافة في النهاية(addBack/insertBack)

- **المفهوم** :إدراج عُقدة جديدة في نهاية القائمة.
- **الخطوات مع مؤشر ذيل:**
  1. إنشاء عُقدة جديدة
  2. جعل مؤشر الذيل الحالي يشير إلى العُقدة الجديدة

3. تحديث مؤشر الذيل ليشير إلى العُقدة الجديدة
4. جعل مؤشر العُقدة الجديدة يشير إلىNULL

- **بدون مؤشر ذيل** :يتطلب اجتياز القائمة بأكملها للوصول إلى آخر عُقدة.
- **التعقيد الزمني** :ثابت مع مؤشر ذيل، خطي بدونه.

2.4.3الإزالة من المقدمة(removeFront)

- **المفهوم** :حذف العُقدة الأولى (الرأس) من القائمة.
- **الخطوات النظرية**:
  1. حفظ مرجع للعُقدة الحالية (الرأس القديم)
  2. تحديث مؤشر الرأس ليشير إلى العُقدة التالية
  3. حذف العُقدة القديمة من الذاكرة
- **الحالة الخاصة** :إذا كانت القائمة تحتوي على عُقدة واحدة، يجب تعيين كل من الرأس والذيل إلى.NULL
- **التعقيد الزمني** :ثابت.

2.4.4الإزالة من النهاية(removeBack)

- **المفهوم** :حذف العُقدة الأخيرة (الذيل) من القائمة.
- **التحدي الرئيسي** :يتطلب الوصول إلى العُقدة قبل الأخيرة لتحديث مؤشرها إلى.NULL
- **الخطوات النظرية**:
  1. اجتياز القائمة حتى الوصول إلى العُقدة قبل الأخيرة
  2. حذف العُقدة الأخيرة
  3. جعل مؤشر العُقدة قبل الأخيرة يشير إلىNULL
  4. تحديث مؤشر الذيل
- **التعقيد الزمني** :خطي، يتطلب اجتياز القائمة بأكملها.

2.4.5البحث(Search)

- **المفهوم** :إيجاد عُقدة تحتوي على قيمة محددة.
- **الآلية** :الاجتياز التسلسلي من الرأس مع مقارنة كل عُقدة بالقيمة المطلوبة.

- **التعقيد الزمني** :خطي في أسوأ حالة (العنصر في النهاية أو غير موجود).

2.5العمومية(Generic Linked Lists)

- **المفهوم** :تصميم القائمة لتعمل مع أي نوع بيانات وليس نوعًا محددًا فقط.
- **الآلية** :استخدام القوالب (Templates) في C++ أو المُعرِّفات العامة (Generics)في لغات أخرى.
- **الفائدة** :إعادة استخدام نفس التركيب البنيوي لأنواع بيانات مختلفة دون تكرار الكود.

3.القوائم المتصلة المزدوجة(Doubly Linked Lists)

3.1التركيب النظري

- **المفهوم الأساسي** :قائمة مُحسَّنة تسمح بالاجتياز في كلا الاتجاهين (الأمام والخلف).
- **بنية العُقدة** :تتكون من:
  - حقل البيانات(data/element)
  - مؤشر أمامي (next) يشير إلى العُقدة التالية
  - مؤشر خلفي (prev/previous) يشير إلى العُقدة السابقة
- **الروابط الثنائية** :كل عُقدة مرتبطة بعُقدتين: السابقة واللاحقة.

3.2المزايا النظرية الرئيسية

- **الاجتياز ثنائي الاتجاه** :يمكن الانتقال للأمام والخلف في القائمة.
- **كفاءة الحذف من أي موضع** :يمكن حذف عُقدة معينة دون اجتياز القائمة من البداية.
- **الوصول السريع للعُقد المجاورة** :كل عُقدة لديها وصول مباشر للعُقدتين المجاورتين.
- **تبسيط بعض العمليات** :عمليات مثل الحذف والإدراج في مواضع محددة تصبح أكثر مباشرة.

3.3القيود النظرية

- **استخدام ذاكرة أكبر** :كل عُقدة تحتاج مؤشر إضافي(prev) ، مما يضاعف مساحة المؤشرات.
- **تعقيد الصيانة** :تحديث الروابط يتطلب عناية أكبر للحفاظ على تماسك البنية.
- **عدد أكبر من عمليات التحديث** :بعض العمليات تتطلب تحديث عدد أكبر من المؤشرات.

3.4العُقد الحارسة(Sentinel Nodes)

3.4.1المفهوم والغرض

- **التعريف** :عُقد خاصة "وهمية" أو "حارسة" لا تخزن بيانات فعلية، توضع في طرفي القائمة.
- **العُقد المستخدمة:**
  - **عُقدة الرأس الحارسة :(Header)** توضع قبل أول عُقدة حقيقية
  - **عُقدة الذيل الحارسة :(Trailer)** توضع بعد آخر عُقدة حقيقية

3.4.2الفوائد النظرية

- **تبسيط معالجة الحالات الخاصة** :تلغي الحاجة للتحقق من القائمة الفارغة أو الحدود الطرفية.
- **توحيد العمليات** :جميع عمليات الإدراج والحذف تتم بنفس الطريقة دون استثناءات.
- **الوصول السريع** :توفر وصولاً مباشرًا ومتسقًا إلى بداية ونهاية القائمة.

3.4.3بنية القائمة الفارغة مع الحارسات

- في القائمة الفارغة:
  - `header->next` يشير إلى `trailer`
  - `trailer->prev` يشير إلى `header`
- لا توجد عُقد بيانات حقيقية بين الحارستين.

3.5العمليات الأساسية

3.5.1الإدراج(Insertion)

- **المفهوم العام** :إدراج عُقدة جديدة (z) بين عُقدتين موجودتين (predecessor)و(successor).
- **الخطوات النظرية**)تحديث أربعة مؤشرات:(
  1. جعل `z->prev` يشير إلى العُقدة السابقة(predecessor)
  2. جعل `z->next` يشير إلى العُقدة التالية(successor)
  3. جعل `predecessor->next` يشير إلى العُقدة الجديدة(z)
  4. جعل `successor->prev` يشير إلى العُقدة الجديدة(z)
- **ترتيب العمليات** :يُفضل تحديث مؤشرات العُقدة الجديدة أولاً لتجنب فقدان المراجع.
- **التعقيد الزمني** :ثابت، بغض النظر عن حجم القائمة.

3.5.2الإزالة(Removal)

- **المفهوم** :حذف عُقدة معينة (v) من القائمة عن طريق "ربط" العُقدتين المجاورتين لها مباشرة.
- **الخطوات النظرية**)تحديث مؤشرين:(
  1. جعل `u->next`(العُقدة السابقة) يشير إلى) wالعُقدة التالية(
  2. جعل `w->prev`(العُقدة التالية) يشير إلى) uالعُقدة السابقة(
  3. حذف العُقدة v من الذاكرة
- **النتيجة** :العُقدة v تصبح "معزولة" أو "مفصولة (linked out) "من القائمة.
- **التعقيد الزمني** :ثابت، إذا كان لدينا مؤشر مباشر للعُقدة المراد حذفها.

3.5.3الإضافة في البداية

- **باستخدام الحارسات** :إدراج بين header والعُقدة الأولى الفعلية(header->next).
- **بدون تحقق من حالات خاصة** :نفس خطوات الإدراج العامة.

3.5.4 الإضافة في النهاية

- **باستخدام الحارسات**: إدراج بين العُقدة الأخيرة الفعلية (trailer->prev) و trailer.
- **التناسق**: نفس الآلية دون حاجة للتمييز عن الإضافة في البداية.

3.5.5 الحذف من النهاية

- **الميزة الرئيسية**: على عكس القائمة الأحادية، الحذف من النهاية يتم بتعقيد زمني ثابت.
- **السبب**: الوصول المباشر للعُقدة قبل الأخيرة عبر `trailer->prev->prev`.

---

4. القوائم المتصلة الدائرية (Circularly Linked Lists)

4.1 التركيب النظري

- **المفهوم الأساسي**: قائمة تستخدم نفس بنية العُقد الأحادية، لكن العُقد مرتبطة في شكل دائري (حلقة).
- **البنية الدائرية**: آخر عُقدة لا تشير إلى NULL ، بل تشير إلى أول عُقدة، مما يُشكل دورة مغلقة.
- **غياب البداية والنهاية الواضحة**: لا يوجد رأس أو ذيل بالمعنى التقليدي.
- **الاجتياز اللانهائي**: يمكن الاستمرار في الاجتياز بشكل دائري دون الوصول إلى نهاية (NULL).

4.2 المؤشر (Cursor)

- **التعريف**: عُقدة خاصة مُعيَّنة تعمل كنقطة مرجعية أو نقطة بداية للتعامل مع القائمة.
- **الوظيفة**: يوفر نقطة ثابتة للوصول إلى القائمة الدائرية وتنفيذ العمليات عليها.
- **المرونة**: يمكن تحريك المؤشر للإشارة إلى أي عُقدة في القائمة.

4.3المصطلحات الخاصة

4.3.1الخلفية(Back)

- **التعريف** :العُقدة التي يشير إليها cursor مباشرة.
- **الوصول** `cursor->data`: أو القيمة المخزنة في العُقدة المُشار إليها.
- **الموضع** :العُقدة "الحالية" أو "عند" المؤشر.

4.3.2المقدمة(Front)

- **التعريف** :العُقدة التي تلي cursor مباشرة في الترتيب الدائري.
- **الوصول**.`cursor->next->data` :
- **الموضع** :العُقدة التالية للمؤشر في اتجاه الدوران.

4.4العلاقة بالقائمة الأحادية

- **التصور النظري** :إذا تم "قطع" الرابط بين cursor والعُقدة التالية لها، تتحول القائمة إلى قائمة أحادية عادية.
- **البنية الناتجة** :قائمة تمتد من عُقدة Front (الرأس) إلى عُقدة Back (الذيل).
- **الاستنتاج** :القائمة الدائرية يمكن اعتبارها قائمة أحادية تم "إغلاقها" بربط النهاية بالبداية.

4.5العمليات الأساسية

4.5.1التقدم(Advance)

- **المفهوم** :تحريك cursor إلى العُقدة التالية في القائمة الدائرية.
- **الآلية**.`cursor = cursor->next` :
- **التأثير** :ما كان "front" يصبح"back" ، والعُقدة التي تليه تصبح "front" الجديدة.
- **الاستخدام** :للدوران عبر القائمة والوصول إلى عُقد مختلفة.

4.5.2 الإضافة (Add)

- **المفهوم**: إدراج عُقدة جديدة مباشرة بعد cursor.
- **الخطوات في قائمة فارغة:**
  1. إنشاء عُقدة جديدة
  2. جعل مؤشرها يشير إلى نفسها (لتشكيل دائرة من عُقدة واحدة)
  3. جعل cursor يشير إلى العُقدة الجديدة
- **الخطوات في قائمة غير فارغة:**
  1. إنشاء عُقدة جديدة
  2. جعل مؤشر العُقدة الجديدة يشير إلى `cursor->next` (Front القديمة)
  3. جعل `cursor->next` يشير إلى العُقدة الجديدة
- **النتيجة**: العُقدة الجديدة تصبح Front الجديدة.
- **التعقيد الزمني**: ثابت.

4.5.3 الإزالة (Remove)

- **المفهوم**: حذف العُقدة التي تلي cursor مباشرة (Front).
- **الخطوات في قائمة بعُقدة واحدة:**
  1. حذف العُقدة الوحيدة
  2. جعل cursor يساوي NULL (القائمة تصبح فارغة)
- **الخطوات في قائمة بعدة عُقد:**
  1. حفظ مرجع للعُقدة المراد حذفها (cursor->next)
  2. جعل cursor->next يشير إلى العُقدة التي تلي العُقدة المحذوفة
  3. حذف العُقدة المحفوظة من الذاكرة
- **التعقيد الزمني**: ثابت.

4.6 التحقق من القائمة الفارغة

- **الشرط**: القائمة فارغة عندما يكون `cursor == NULL`.
- **الفرق عن القوائم الأخرى**: لا يوجد مفهوم NULL في نهاية القائمة عندما تكون غير فارغة، فقط في حالة الفراغ الكامل.

4.7التطبيقات النظرية

- **الجدولة الدائرية :(Round-Robin Scheduling)** توزيع الموارد بالتناوب بين العمليات.
- **القوائم الدائمة :**تمثيل البيانات التي لا تحتاج بداية أو نهاية واضحة.
- **الألعاب متعددة اللاعبين :**تمثيل دور اللاعبين الذي يدور بشكل دائري.
- **البُنى الحلقية :**أي هيكل بيانات يتطلب تكرار دوري.

5.المقارنة النظرية الشاملة

5.1التعقيد الزمني للعمليات

القائمة الدائرية	القائمة المزدوجة	القائمة الأحادية	العملية
O(n)	O(n)	O(n)	الوصول بالفهرس
O(n)	O(n)	O(n)	البحث
-	O(1)	O(1)	الإضافة في البداية
O(1)	O(1)	O(1)*	الإضافة في النهاية
-	O(1)	O(1)	الحذف من البداية
O(1)	O(1)	O(n)	الحذف من النهاية
-	O(1)**	O(n)**	الإدراج في الوسط
-	O(1)**	O(n)**	الحذف من الوسط

*بشرط وجود مؤشر tail

**بشرط معرفة موقع العُقدة مسبقاً

5.2التعقيد المكاني

المساحة الإضافية الكلية	مساحة المؤشرات لكل عُقدة	نوع القائمة
O(n)	مؤشر واحد	الأحادية
O(2n)	مؤشران	المزدوجة
O(n)	مؤشر واحد	الدائرية

5.3المزايا والعيوب النسبية

القائمة الأحادية:

- ✓بساطة التركيب والتنفيذ
- ✓استخدام أقل للذاكرة
- ✗اجتياز أحادي الاتجاه فقط
- ✗حذف بطيء من النهاية

القائمة المزدوجة:

- ✓اجتياز ثنائي الاتجاه
- ✓عمليات حذف وإدراج فعالة في أي موضع
- ✓مرونة أكبر في العمليات
- ✗استخدام ضعف الذاكرة للمؤشرات
- ✗تعقيد أكبر في الصيانة

القائمة الدائرية:

- ✓مثالية للبيانات الدورية
- ✓لا حاجة لتتبع بداية أو نهاية
- ✓الاجتياز المستمر الممكن
- ✗خطر الحلقات اللانهائية
- ✗أقل شيوعاً في الاستخدام

6.الخصائص النظرية العامة للقوائم المتصلة

6.1مقارنة مع المصفوفات

المزايا على المصفوفات:

- حجم ديناميكي غير محدود مسبقاً
- إدراج وحذف فعال دون إزاحة العناصر
- استخدام الذاكرة حسب الحاجة الفعلية
- سهولة إعادة الترتيب والدمج

العيوب مقارنة بالمصفوفات:

- عدم وجود وصول عشوائي مباشر
- استخدام ذاكرة إضافية للمؤشرات
- ذاكرة غير متجاورة) أقل كفاءة في الـcache)
- تعقيد أكبر في التنفيذ والصيانة

6.2الاعتبارات النظرية لاختيار النوع المناسب

استخدم القائمة الأحادية عندما:

- العمليات تتركز على بداية القائمة
- الذاكرة محدودة والكفاءة مهمة
- الاجتياز يتم في اتجاه واحد فقط
- البساطة مطلوبة

استخدم القائمة المزدوجة عندما:

- الحاجة للاجتياز في الاتجاهين
- عمليات الحذف والإدراج متكررة في مواضع مختلفة
- الأداء أهم من استخدام الذاكرة
- تنفيذ هياكل بيانات معقدة) مثلDeque)

استخدم القائمة الدائرية عندما:

- البيانات ذات طبيعة دورية
- الحاجة للدوران المستمر عبر العناصر
- تطبيقات الجدولة الدائرية
- عدم وجود بداية أو نهاية طبيعية للبيانات

6.3الملاحظات النهائية

القوائم المتصلة توفر بديلاً مرنًا وديناميكيًا للمصفوفات، حيث تتغلب على قيود الحجم الثابت وتكلفة الإزاحة في عمليات الإدراج والحذف. كل نوع من القوائم المتصلة يقدم مقايضات مختلفة بين الكفاءة واستخدام الذاكرة والمرونة، مما يجعل اختيار النوع المناسب معتمدًا على متطلبات التطبيق المحددة وطبيعة العمليات المتوقعة على البيانات.

الاستدعاء الذاتي (Recursion)

يُعد الاستدعاء الذاتي من التقنيات البرمجية الأساسية التي توفر بديلاً قوياً وأنيقاً للحلقات التكرارية (Loops) في معالجة المشكلات ذات البنية المتكررة. يستعرض هذا القسم المفاهيم النظرية الأساسية للاستدعاء الذاتي وأنواعه المختلفة مع تطبيقاته العملية.

1.المفهوم الأساسي والتعريف

1.1التعريف النظري

- **الاستدعاء الذاتي** :هو أسلوب برمجي تستدعي فيه الدالة نفسها بشكل مباشر أو غير مباشر لحل مشكلة معينة.
- **الاستدعاء المباشر** :عندما تستدعي الدالة نفسها مباشرة في جسم تعريفها.
- **الاستدعاء غير المباشر** :عندما تستدعي الدالة A دالة أخرىB ، والتي بدورها تستدعي الدالة A مرة أخرى.

1.2الغرض والميزة الأساسية

- **الأناقة والوضوح** :توفر طريقة أكثر وضوحاً وقابلية للقراءة مقارنة بالحلول التكرارية المعقدة.
- **استغلال البنية المتكررة** :تعمل بشكل طبيعي مع المشكلات التي لها بنية متكررة أو تعريف ذاتي التشابه.
- **تجنب التعقيد** :تلغي الحاجة لاستخدام حلقات متداخلة معقدة وتحليل حالات متعددة.
- **التعبير الرياضي** :تعكس التعريفات الرياضية العودية بشكل مباشر.

1.3مجالات التطبيق الرئيسية

1.3.1الهياكل البيانية المتكررة

- **أنظمة الملفات** :المجلدات تحتوي على ملفات ومجلدات أخرى (بنية شجرية).
- **الأشجار** :كل عقدة في الشجرة هي جذر لشجرة فرعية.
- **القوائم المتصلة** :كل عقدة تشير إلى قائمة فرعية.

1.3.2 العمليات الحسابية

- **حساب المضروب (Factorial):** تعريف رياضي عودي طبيعي.
- **أرقام فيبوناتشي (Fibonacci):** كل رقم هو مجموع الرقمين السابقين.
- **القوى الأسية:** حساب قوى الأعداد بطريقة تقسيم وغزو.

1.3.3 الخوارزميات المتقدمة

- **خوارزميات الفرز:** Merge Sort، Quick Sort.
- **خوارزميات البحث:** Binary Search على الأشجار.
- **تقنيات التراجع (Backtracking):** حل الألغاز والمشكلات التوافقية.

1.3.4 معالجة النصوص

- **المحللات النحوية (Parsers):** تحليل بنية الجمل في اللغات البرمجية.
- **تحليل التعبيرات الرياضية:** معالجة الأقواس والعمليات المتداخلة.

---

2. المكونات الأساسية للتعريف العودي

2.1 حالات الأساس (Base Cases)

2.1.1 التعريف والوظيفة

- **المفهوم:** الحالات التي تُحل بشكل مباشر دون الحاجة لاستدعاء عودي.
- **الوظيفة الرئيسية:** توفير نقطة إيقاف للاستدعاءات العودية لمنع التكرار اللانهائي.
- **الخصائص:** يجب أن تكون بسيطة ومحددة بوضوح، ويمكن حسابها مباشرة.

2.1.2 الأهمية النظرية

- **ضمان الإنهاء:** بدون حالة أساسية، تستمر الدالة في الاستدعاء الذاتي إلى ما لا نهاية.
- **الأساس المنطقي:** تمثل أبسط صورة للمشكلة التي لا تحتاج تقسيم أو تصغير.

- **نقطة البداية للعودة** :بعد الوصول لحالة الأساس، تبدأ الدالة في إرجاع النتائج للاستدعاءات السابقة.

2.1.3متطلبات التصميم

- **التغطية الشاملة** :يجب أن تغطي جميع الحالات الطرفية الممكنة.
- **القابلية للوصول** :كل تسلسل من الاستدعاءات يجب أن يصل لحالة أساسية في النهاية.
- **البساطة** :الحل يجب أن يكون واضحاً ومباشراً.

2.2الحالات العودية(Recursive Cases)

2.2.1التعريف والآلية

- **المفهوم** :الحالات التي تُحل عن طريق تقسيم المشكلة إلى مشكلات أصغر من نفس النوع.
- **الآلية** :تستدعي الدالة نفسها بمعاملات مُعدَّلة تمثل نسخة أصغر من المشكلة الأصلية.
- **التقدم الإلزامي** :كل استدعاء عودي يجب أن يقرّب المشكلة من حالة الأساس.

2.2.2شروط الصحة النظرية

- **تصغير المشكلة** :المعاملات المُمررة يجب أن تكون "أصغر" أو "أبسط" من المعاملات الأصلية.
- **تجنب الدورية** :**(Circularity)** لا يجوز أن تؤدي الاستدعاءات إلى نفس الحالة مراراً.
- **الاقتراب من الأساس** :التأكد من أن التصغير المتكرر يؤدي حتماً لحالة أساسية.

2.2.3أنماط التصغير الشائعة

- **التصغير العددي** :تقليل قيمة عددية.$(n \rightarrow n-1)$
- **التقسيم إلى نصفين** :تقسيم المشكلة لنصفين متساويين تقريباً.
- **الإزالة التدريجية** :إزالة عنصر واحد من مجموعة (قائمة، مصفوفة).

- **التفكيك الهيكلي**: تفكيك بنية معقدة إلى أجزاء أصغر.

2.3 العلاقة بين المكونات

- **التكامل**: حالة الأساس والحالة العودية يجب أن تكملا بعضهما لتغطية جميع الحالات الممكنة.
- **الانتقال السلس**: يجب أن يكون الانتقال من الحالات العودية إلى حالة الأساس واضحاً ومحدداً.
- **عدم التداخل**: يُفضل ألا تتداخل شروط حالة الأساس مع شروط الحالات العودية.

---

3. أنواع الاستدعاء الذاتي

3.1 الاستدعاء الذاتي الخطي (Linear Recursion)

3.1.1 التعريف النظري

- **المفهوم**: الدالة تقوم باستدعاء عودي واحد فقط كحد أقصى في كل مرة يتم فيها تنفيذها.
- **البنية**: بعد اختبار حالة الأساس، يتم إجراء استدعاء واحد على نسخة أصغر من المشكلة.
- **التسلسل**: الاستدعاءات تشكل سلسلة خطية من التنفيذات.

3.1.2 الخصائص النظرية

- **عدد الاستدعاءات**: يتناسب خطياً مع حجم المدخل (O(n) استدعاء.
- **عمق الاستدعاء**: يساوي عدد الخطوات اللازمة للوصول لحالة الأساس.
- **استخدام الذاكرة**: يتطلب مساحة O(n) على المكدس.(Stack)
- **البساطة**: أبسط أشكال الاستدعاء الذاتي وأسهلها فهماً.

3.1.3 الأمثلة والتطبيقات الأساسية

1. حساب المضروب (Factorial)

- **التعريف الرياضي** :$n! = n \times (n-1)!$ حيث $0! = 1$
- **الاستخدام** :حسابات التباديل والتوافيق، نظرية الأعداد.
- **الحالة الأساسية** :عندما $n = 0$ أو $n = 1$
- **الحالة العودية** :ضرب n في مضروب(n-1)

2.جمع عناصر مصفوفة

- **المفهوم** :جمع كل عناصر مصفوفة بشكل تكراري.
- **الاستخدام** :معالجة البيانات، الإحصاء.
- **الآلية** :جمع العنصر الأول مع مجموع بقية العناصر.

3.البحث الخطي العودي

- **المفهوم** :البحث عن عنصر في قائمة أو مصفوفة.
- **الآلية** :فحص العنصر الأول، ثم البحث في بقية القائمة.
- **الحالة الأساسية** :العثور على العنصر أو وصول نهاية القائمة.

4.حساب القوى(Power Calculation)

- **التعريف** :$x^n = x \times x^{(n-1)}$
- **الاستخدام** :العمليات الرياضية، التشفير.
- **التحسين** :يمكن تحسينه باستخدام العودية الثنائية.

5.عكس سلسلة نصية(String Reversal)

- **المفهوم** :عكس ترتيب الأحرف في نص.
- **الآلية** :وضع الحرف الأول في النهاية وعكس البقية.

6.طباعة الأرقام(Printing Sequence)

- **التطبيقات** :طباعة الأعداد من 1 إلى n أو من n إلى 1.
- **الاستخدام التعليمي** :فهم تدفق الاستدعاءات العودية.

7.اجتياز القائمة المتصلة

- **المفهوم** :زيارة كل عُقدة في قائمة متصلة.
- **الآلية** :معالجة العُقدة الحالية ثم الانتقال للعُقدة التالية عودياً.

3.1.4 المزايا والقيود

المزايا:

- سهولة الفهم والتنفيذ
- تطابق مباشر مع التعريفات الرياضية
- كود واضح ومقروء

القيود:

- استخدام ذاكرة كبير على المكدس
- قد يكون بطيئاً مقارنة بالحلقات للمشكلات البسيطة
- خطر تجاوز سعة المكدس (Stack Overflow) مع المدخلات الكبيرة

3.2 الاستدعاء الذاتي الذيلي (Tail Recursion)

3.2.1 التعريف الدقيق

- **المفهوم** :شكل خاص من الاستدعاء الخطي حيث الاستدعاء العودي هو **آخر عملية تماماً** في الدالة.
- **الشرط الأساسي** :لا يتم إجراء أي عملية حسابية أو معالجة بعد عودة الاستدعاء العودي.
- **الفرق الجوهري** :القيمة المُرجعة من الاستدعاء العودي تُرجع مباشرة دون تعديل.

3.2.2 الخصائص النظرية المميزة

- **إمكانية التحويل** :يمكن تحويل أي دالة ذيلية إلى حلقة تكرارية بسهولة.
- **الكفاءة المحتملة** :المترجمات الحديثة يمكنها تحسين الاستدعاء الذيلي (Tail Call Optimization).

- **توفير الذاكرة** :مع التحسين، لا تحتاج لحفظ إطارات استدعاء متعددة على المكدس.
- **التعقيد المكاني** :يمكن تقليله من O(n) إلى O(1) مع التحسين.

3.2.3 الأمثلة والتطبيقات

1. المضروب الذيلي (Tail-Recursive Factorial)

- **الآلية** :استخدام معامل إضافي (accumulator) لحفظ النتيجة الجزئية.
- **الميزة** :تجنب الضرب بعد العودة من الاستدعاء.
- **الفرق** :النتيجة تُحسب أثناء الذهاب للأمام، لا أثناء العودة.

2. جمع الأعداد من 1 إلى n

- **الطريقة التقليدية** - n + sum(n-1) :ليست ذيلية.
- **الطريقة الذيلية** :استخدام accumulator لتجميع المجموع تدريجياً.

3. البحث في القائمة المتصلة

- **الاستخدام** :البحث عن عنصر معين.
- **الميزة** :لا يحتاج عمليات بعد العودة، فقط return النتيجة.

4. معالجة الملفات الكبيرة

- **التطبيق** :قراءة وتحليل ملفات سطراً بسطر.
- **الفائدة** :كفاءة في الذاكرة للملفات الضخمة.

3.2.4 التحويل من عودي إلى تكراري

- **المبدأ** :استبدال الاستدعاء العودي بحلقة while أو for.
- **الخطوات**:
  1. تحويل المعاملات إلى متغيرات محلية
  2. استبدال الاستدعاء العودي بحلقة
  3. تحديث المتغيرات في كل تكرار

- **النتيجة** :كود أكثر كفاءة بنفس المنطق.

3.2.5المزايا والاستخدام الأمثل

المزايا:

- كفاءة عالية في الذاكرة (مع التحسين)
- تجنبStack Overflow
- سهولة التحويل للتكرارية

الاستخدام الأمثل:

- العمليات التراكمية (جمع، ضرب)
- معالجة البيانات التسلسلية
- الخوارزميات التي لا تحتاج للعودة بنتائج

3.3الاستدعاء الذاتي الثنائي(Binary Recursion)

3.3.1التعريف النظري

- **المفهوم** :الدالة تقوم بإجراء استدعاءين عوديين في كل مرة يتم فيها تنفيذها.
- **البنية** :تقسيم المشكلة إلى مشكلتين فرعيتين متشابهتين أو متماثلتين.
- **التسلسل** :الاستدعاءات تشكل بنية شجرية ثنائية.

3.3.2الخصائص النظرية

- **عدد الاستدعاءات** :يمكن أن ينمو بشكل أسي ((O(2^n) في أسوأ الحالات.
- **البنية الشجرية** :شجرة الاستدعاءات لها فرعان عند كل عُقدة.
- **عمق الشجرة** :يعتمد على كيفية تقسيم المشكلة.
- **التعقيد المكاني** O(h) :حيث h هو ارتفاع شجرة الاستدعاءات.

3.3.3 الأمثلة والتطبيقات الرئيسية

1. أرقام فيبوناتشي (Fibonacci Numbers)

- **التعريف**: $F(n) = F(n-1) + F(n-2)$، حيث $F(0)=0$ و $F(1)=1$
- **الاستخدام**: الرياضيات، تحليل الخوارزميات، الطبيعة.
- **المشكلة**: التداخل الشديد في الحسابات (Overlapping Subproblems).
- **التعقيد** - $O(2^n)$: غير فعال للأعداد الكبيرة.
- **الحل**: يمكن تحسينه باستخدام البرمجة الديناميكية أو تحويله لخطي.

2. جمع عناصر مصفوفة (Binary Sum)

- **الآلية**: تقسيم المصفوفة إلى نصفين، جمع كل نصف عودياً، ثم جمع النتيجتين.
- **الاستخدام**: تقنية "فرّق تسُد (Divide and Conquer)."
- **الميزة**: عمق استدعاء أقل $O(\log n)$ مقارنة بالخطي $O(n)$.
- **التطبيقات**: أساس لخوارزميات الفرز والبحث المتقدمة.

3. خوارزمية الفرز بالدمج (Merge Sort)

- **المفهوم**: تقسيم المصفوفة إلى نصفين، فرز كل نصف، ثم دمجهما.
- **التعقيد الزمني** - $O(n \log n)$: من أكفأ خوارزميات الفرز.
- **الاستخدام**: فرز كميات كبيرة من البيانات.
- **الميزة**: أداء مستقر بغض النظر عن توزيع البيانات.

4. خوارزمية الفرز السريع (Quick Sort)

- **الآلية**: اختيار pivot ، تقسيم البيانات حوله، فرز الجزئين.
- **التعقيد** $O(n \log n)$: في المتوسط، $O(n^2)$ في أسوأ حالة.
- **الاستخدام**: أحد أكثر خوارزميات الفرز شيوعاً.

5. البحث الثنائي على الأشجار (Binary Tree Traversal)

- **أنواع الاجتياز**:
  - **Preorder**: الجذر ثم اليسار ثم اليمين

- **Inorder**: اليسار ثم الجذر ثم اليمين
- **Postorder**: اليسار ثم اليمين ثم الجذر
- **الاستخدام**: معالجة البيانات الهرمية، تحليل التعبيرات.

6. حساب القوى الأسية السريع (Fast Exponentiation)

- **المفهوم**: $x^n = (x^{(n/2)})^2$ للأعداد الزوجية
- **التعقيد**: $O(\log n)$ بدلاً من $O(n)$
- **الاستخدام**: التشفير، العمليات الرياضية الكبيرة.

7. أبراج هانوي (Towers of Hanoi)

- **الوصف**: نقل n قرص من عمود لآخر باستخدام عمود مساعد.
- **القواعد**: قرص واحد في المرة، لا قرص كبير فوق صغير.
- **التعقيد**: $O(2^n)$ - عدد الحركات = $2^n - 1$
- **الاستخدام التعليمي**: مثال كلاسيكي للاستدعاء الذاتي.

3.3.4 مشكلة التداخل (Overlapping Subproblems)

- **التعريف**: نفس المشكلة الفرعية تُحل عدة مرات في استدعاءات مختلفة.
- **المثال الرئيسي**: فيبوناتشي $F(3)$ - يُحسب مرات متعددة عند حساب $F(5)$.
- **التأثير**: نمو أسي في عدد الاستدعاءات وهدر كبير في الموارد.
- **الحلول**:
  - **Memoization**: حفظ النتائج المحسوبة في جدول
  - **التحويل للخطي**: إعادة صياغة المشكلة
  - **البرمجة الديناميكية**: بناء الحل من الأسفل للأعلى

3.3.5 المزايا والقيود

المزايا:

- مناسب لتقنية "فرّق تسُد"
- عمق استدعاء أقل ($O(\log n)$) في بعض الحالات

- يعكس البنية الشجرية الطبيعية للمشكلة

القيود:

- نمو أسي محتمل في عدد الاستدعاءات
- مشكلة التداخل في بعض التطبيقات
- استهلاك كبير للموارد إذا لم يتم التحسين

3.4الاستدعاء الذاتي المتعدد(Multiple Recursion)

3.4.1التعريف النظري

- **المفهوم** :تعميم للاستدعاء الثنائي، حيث تقوم الدالة بأكثر من استدعاءين عوديين.
- **البنية** :شجرة الاستدعاءات تكون متشعبة بأكثر من فرعين عند كل عُقدة.
- **المرونة** :عدد الاستدعاءات قد يكون ثابتاً أو متغيراً حسب المشكلة.

3.4.2الخصائص النظرية

- **التعقيد** :عادةً O(k^n) حيث k هو عدد الاستدعاءات
- **البنية الشجرية** :شجرة) k-ary متعددة الفروع(
- **الاستخدام** :المشكلات التوافقية المعقدة
- **المساحة** :تتطلب ذاكرة كبيرة لشجرة الاستدعاءات

3.4.3الأمثلة والتطبيقات الرئيسية

1.توليد التباديل(Permutations)

- **المفهوم** :توليد جميع الترتيبات الممكنة لمجموعة من العناصر.
- **الآلية** :لكل موضع، جرّب كل العناصر غير المستخدمة.
- **عدد الاستدعاءات** n :استدعاء في المستوى الأول، (n-1)في الثاني، وهكذا.
- **التعقيد** - O(n!) :مضروبي.
- **الاستخدام** :التشفير، تحليل الخوارزميات، الألعاب.

2.توليد التوافيق(Combinations)

- **المفهوم** :اختيار k عنصر من n عنصر بجميع الطرق الممكنة.
- **التعقيد** O(C(n,k)) :حيث C هي دالة التوافيق.
- **الاستخدام** :الإحصاء، نظرية الاحتمالات، التحليل التوافقي.

3.مسألة الملكات الثماني(N-Queens Problem)

- **الوصف** :وضع N ملكة على رقعة شطرنج N×N بحيث لا تهاجم إحداها الأخرى.
- **الآلية** :في كل صف، جرّب وضع الملكة في كل عمود ممكن.
- **التقنية** - Backtracking :التراجع عند الوصول لحل غير صالح.
- **الاستخدام** :الذكاء الاصطناعي، حل القيود.

4.مسألة مجموع المجموعات الجزئية(Subset Sum)

- **المفهوم** :إيجاد مجموعة جزئية من أرقام مجموعها يساوي قيمة معينة.
- **الاستدعاءات** :لكل عنصر، استدعاءان: تضمينه أو استبعاده.
- **التطبيق** :التشفير، تحسين الموارد، الألغاز.

5.حل متاهة(Maze Solving)

- **الوصف** :إيجاد مسار من نقطة البداية للنهاية في متاهة.
- **الاستدعاءات** :أربعة استدعاءات (أعلى، أسفل، يمين، يسار).
- **التقنية** Backtracking :مع تعليم المسارات المزارة.

6.مسألة حقيبة الظهر(Knapsack Problem)

- **الوصف** :اختيار عناصر بأقصى قيمة دون تجاوز وزن معين.
- **الاستخدام** :تحسين الموارد، التخطيط اللوجستي.

7.توليد سلاسل الأقواس(Parentheses Generation)

- **المفهوم** :توليد جميع التوليفات الصحيحة لـ n زوج من الأقواس.

- **الاستدعاءات** :استدعاءان (إضافة قوس مفتوح أو مغلق).
- **الاستخدام** :محللات اللغات البرمجية، التعبيرات الرياضية.

8.رسم منحنيات فراكتال(Fractal Curves)

- **الأمثلة**Sierpinski Triangle ،: Koch Snowflake
- **الآلية** :رسم جزء، ثم استدعاءات متعددة لرسم أجزاء أصغر مشابهة.
- **الاستخدام** :الرسوميات الحاسوبية، الفن الرقمي، الهندسة الفركتالية.

3.4.4تقنية التراجع(Backtracking)

- **المفهوم** :استكشاف جميع الحلول الممكنة مع التراجع عند الوصول لطريق مسدود.
- **الآلية:**
  1. بناء الحل تدريجياً (خطوة بخطوة)
  2. في كل خطوة، جرّب جميع الخيارات الممكنة
  3. إذا وصلت لحل غير صالح، تراجع وجرّب خياراً آخر
  4. استمر حتى إيجاد حل أو استنفاذ الخيارات
- **التطبيقات الرئيسية:**
  - الألغاز(Sudoku) ، (Crossword
  - الألعاب(Chess) ، (Go
  - مشكلات التحسين
  - توليد الحلول التوافقية

3.4.5المزايا والقيود

المزايا:

- حل شامل للمشكلات التوافقية المعقدة
- استكشاف كل الحلول الممكنة
- مرونة في التعامل مع القيود المختلفة
- يعكس البنية الطبيعية للمشكلات متعددة الخيارات

القيود:

- تعقيد زمني عالي جداً (أسي أو مضروبي)
- غير عملي للمدخلات الكبيرة
- استهلاك كبير للذاكرة
- يحتاج تقنيات تحسين (pruning) لتكون عملية

4.تتبع الاستدعاء الذاتي(Recursion Trace)

4.1المفهوم والغرض

- **التعريف** :أداة مرئية لتحليل وتصوير تنفيذ الدالة العودية خطوة بخطوة.
- **الوظيفة** :فهم تدفق الاستدعاءات وكيفية بناء الحل من الحالات الأساسية.
- **الأهمية** :أداة تعليمية وتحليلية أساسية لفهم العودية.

4.2آلية التتبع

4.2.1المكونات البصرية

- **الصناديق** :**(Boxes)** كل صندوق يمثل استدعاء واحد للدالة.
- **المعاملات** :تُكتب قيم المعاملات داخل أو فوق كل صندوق.
- **الأسهم** :تشير إلى اتجاه الاستدعاءات (للأسفل) والعودة (للأعلى).
- **القيم المُرجعة** :تُكتب على أسهم العودة.

4.2.2مراحل التتبع

- **مرحلة الذهاب** :**(Downward Phase)** الاستدعاءات تتعمق حتى الوصول لحالة الأساس.
- **حالة الأساس** :الصندوق الذي يُحل مباشرة دون استدعاء آخر.
- **مرحلة العودة** :**(Upward Phase)** النتائج تُرجع وتُدمج في كل مستوى.

4.3 أنواع التتبع

4.3.1 التتبع الخطي

- **البنية**: سلسلة رأسية من الصناديق.
- **المثال**: تتبع factorial(4) يظهر 4 صناديق متتالية.
- **التحليل**: سهل المتابعة، يوضح العمق بشكل واضح.

4.3.2 التتبع الشجري

- **البنية**: شجرة من الصناديق، كل عُقدة لها فروع.
- **المثال**: تتبع fibonacci(5) يظهر شجرة متفرعة.
- **التحليل**: يكشف عن التداخل وإعادة الحساب.

4.3.3 التتبع المتعدد

- **البنية**: شجرة k-ary معقدة.
- **المثال**: تتبع توليد التباديل.
- **التحليل**: يوضح النمو الأسي في عدد الاستدعاءات.

4.4 التحليل الكمي

4.4.1 حساب عدد الاستدعاءات

- **الخطي**: n استدعاء لمدخل بحجم n.
- **الثنائي البسيط**: 2^(n+1) - 1 استدعاء (فيبوناتشي).
- **الثنائي المُحسَّن**: 2n - 1 استدعاء (مثل الفرز بالدمج).
- **المتعدد**: k^n أو n! حسب طبيعة المشكلة.

4.4.2 حساب عمق الاستدعاء

- **الخطي**: عمق (n = حجم المدخل).
- **الثنائي المتوازن**: عمق $= \log_2(n)$.
- **المتعدد**: يعتمد على استراتيجية التقسيم.

4.4.3 التعقيد المكاني

- **المساحة على المكدس**: تساوي عمق الاستدعاء × حجم كل إطار.
- **الذاكرة الإضافية**: البيانات المحلية في كل استدعاء.
- **إجمالي الذاكرة**: مجموع المكدس والبيانات المساعدة.

4.5 استخدامات التتبع

4.5.1 التعليم والفهم

- تصور كيفية عمل الاستدعاء الذاتي
- فهم متى تُحل المشكلة وكيف تُجمع النتائج
- اكتشاف الأخطاء في المنطق

4.5.2 التحسين

- تحديد الاستدعاءات المُكررة
- اكتشاف فرص استخدام Memoization
- تقدير الأداء والموارد المطلوبة

4.5.3 التصحيح (Debugging)

- تتبع القيم في كل مستوى
- تحديد أين يحدث الخطأ
- فهم سبب عدم الوصول لحالة الأساس

5.المقارنة والاعتبارات العملية

5.1مقارنة الأنواع

النوع	عدد الاستدعاءات	عمق المكدس	التعقيد الزمني	التعقيد المكاني	الأمثلة الرئيسية
Linear	1	O(n)	O(n)	O(n)	Factorial, Sum
Tail	1	O(n) أو O(1)*	O(n)	O(1)*	Tail Factorial
Binary	2	O(log n) - O(n)	O(n) - O(2^n)	O(log n) - O(n)	Fibonacci, Merge Sort
Multiple	k (k > 2)	متفاوت	O(k^n) - O(n!)	متفاوت	-Permutations, N Queens

*مع تحسين المترجم (Tail Call Optimization)

5.2متى تستخدم الاستدعاء الذاتي؟

5.2.1الحالات المثالية

- **البنية الطبيعية المتكررة**: المشكلة تُعرَّف بشكل عودي طبيعياً.
- **الأشجار والرسوم البيانية**: اجتياز ومعالجة الهياكل الهرمية.
- **فرّق تسُد**: تقسيم المشكلة لمشكلات أصغر متشابهة.
- **التراجع**: استكشاف الحلول مع إمكانية التراجع.
- **الوضوح مطلوب**: عندما تكون قابلية القراءة أهم من الأداء الخام.

5.2.2الحالات التي يُفضل تجنبها

- **البيانات الخطية البسيطة**: يمكن حلها بحلقة أبسط.
- **المدخلات الكبيرة جداً**: خطر تجاوز سعة المكدس.
- **متطلبات الأداء الحرجة**: الحلقات عادةً أسرع.

- **الذاكرة محدودة** :الاستدعاء العودي يستهلك مساحة المكدس.

5.3المخاطر والمشاكل الشائعة

5.3.1تجاوز سعة المكدس(Stack Overflow)

- **السبب** :عمق استدعاء كبير جداً يتجاوز سعة المكدس المخصصة.
- **الوقاية:**
  - استخدام الذيلية عندما ممكن
  - وضع حد أقصى لعمق الاستدعاء
  - التحويل للحلقات للمدخلات الكبيرة

5.3.2الحلقة اللانهائية(Infinite Recursion)

- **السبب** :عدم وجود حالة أساسية أو عدم الوصول إليها.
- **الوقاية:**
  - التأكد من وجود حالة أساسية صحيحة
  - ضمان التقدم نحو حالة الأساس في كل استدعاء
  - اختبار الحالات الحدية

5.3.3الأداء الضعيف

- **السبب** :استدعاءات مُكررة.(Overlapping Subproblems)
- **الحلول:**
  - **Memoization**:حفظ النتائج المحسوبة
  - **البرمجة الديناميكية** :بناء الحل من الأسفل
  - **التحويل للخطي** :إعادة صياغة المشكلة

5.4تقنيات التحسين

5.4.1 Memoization

- **المفهوم** :حفظ نتائج الاستدعاءات السابقة في جدول.(cache)
- **الآلية** :قبل الحساب، تحقق من الجدول؛ إذا موجود، أرجعه فوراً.

- **التأثير** :تحويل التعقيد من $O(2^n)$ إلى $O(n)$ في كثير من الحالات.
- **الاستخدام**fibonacci :، مشكلات البرمجة الديناميكية.

5.4.2 Tail Call Optimization

- **المفهوم** :المترجم يحوّل الاستدعاء الذيلي لحلقة تكرارية.
- **المتطلب** :الدالة يجب أن تكون ذيلية فعلاً.
- **التأثير** :تقليل المساحة من $O(n)$ إلى.$O(1)$
- **التوافر** :مدعوم في بعض اللغات والمترجمات.

5.4.3تقليم الشجرة(Pruning)

- **المفهوم** :تجنب استكشاف الفروع التي لن تؤدي لحل صالح.
- **الاستخدام** :خوارزميات التراجع، الذكاء الاصطناعي.
- **التأثير** :تقليل كبير في عدد الاستدعاءات.

5.5العودية vs التكرار(Loops)

5.5.1مزايا العودية

- وضوح الكود وقابلية القراءة
- تطابق مع التعريفات الرياضية
- مناسبة للبنى الهرمية والمتكررة
- تبسيط المنطق المعقد

5.5.2مزايا الحلقات

- كفاءة أكبر في الأداء
- استهلاك أقل للذاكرة
- لا خطر منStack Overflow
- سيطرة أفضل على التكرار

المراجعة المركزة السريعة للفصل الأول

1.المصفوفات - (Arrays) النقاط الحرجة

التعقيدات الزمنية الأساسية

- **الوصول بالفهرس** - O(1): ميزة رئيسية
- **الإضافة/الحذف في البداية/الوسط** - O(n): بسبب الإزاحة
- **الإضافة في النهاية** O(1): إذا يوجد مساحة
- **البحث في مصفوفة غير مرتبة**O(n) :

عمليات الإزاحة (حرجة للامتحان)

الإضافة المرتبة:

- ابدأ الإزاحة من **اليمين إلى اليسار**)من النهاية للموضع المطلوب(
- لتجنب الكتابة فوق البيانات

الحذف:

- ابدأ الإزاحة من **اليسار إلى اليمين**)من الموضع التالي للحذف(
- لملء الفجوة

Insertion Sort - مهم جداً

- **أفضل حالة** - O(n): المصفوفة مرتبة مسبقاً
- **أسوأ حالة** - $O(n^2)$: المصفوفة مرتبة عكسياً
- **متوسط الحالة**$O(n^2)$:
- **المساحة**in-place sorting - O(1) :

المصفوفات ثنائية الأبعاد

- **التعريف**: مصفوفة من المصفوفات `int arr[rows][cols]`
- **الفهرس الأول**: الصف(row)
- **الفهرس الثاني**: العمود(column)
- **التخصيص الديناميكي**: خصص مصفوفة من المؤشرات، ثم كل صف
- **إلغاء التخصيص**: احذف كل صف أولاً، ثم مصفوفة المؤشرات

2.القوائم المتصلة - (Linked Lists) جدول المقارنة الشامل

الفروقات الجوهرية

الخاصية	الأحادية	المزدوجة	الدائرية
عدد المؤشرات/عقدة	1 (next)	2 (next, prev)	1 (next)
نهاية القائمة	NULL	NULL	تشير للبداية
اتجاه الاجتياز	أمامي فقط	أمامي وخلفي	دائري
الحذف من النهاية	O(n)	O(1)	O(1)
المساحة	O(n)	O(2n)	O(n)
الحارسات	لا	نعم(Header/Trailer)	لا

العمليات الحرجة والتعقيدات

القائمة الأحادية:

- `addFront()`: O(1)
- `removeFront()`: O(1)
- `addBack()`: O(1) *بوجود tail pointer*
- `removeBack()`: O(n) ⚠ **نقطة ضعف رئيسية**

القائمة المزدوجة:

- جميع عمليات الإضافة/الحذف ✓ O(1) :
- **الإدراج**: تحديث 4 مؤشرات
- **الحذف**: تحديث 2 مؤشرات فقط

القائمة الدائرية:

- `advance()`: cursor = cursor->next
- `add()`: إضافة بعد cursor (Front الجديدة)
- `remove()`: حذف cursor->next (Front الحالية)
- **تحقق الفراغ**: cursor == NULL

الحارسات - (Sentinels) مهم للمزدوجة

الفائدة الرئيسية:

- إلغاء الحالات الخاصة (القائمة الفارغة، الإضافة في البداية/النهاية)
- **قائمة فارغة**: header->next = trailer, trailer->prev = header
- جميع العمليات موحدة

3.الاستدعاء الذاتي - (Recursion) الجدول الشامل

المكونات الإلزامية

1. **Base Case(s)**: حل مباشر بدون استدعاء

2. **Recursive Case(s):** استدعاء مع **تقدم نحو** Base Case
3. **التقدم الإلزامي:** كل استدعاء يجب أن يقرّب المشكلة من الحل

جدول أنواع العودية - حفظ إلزامي

النوع	عدد الاستدعاءات	التعقيد الزمني	التعقيد المكاني	الأمثلة الرئيسية
الخطي	1	O(n)	O(n)	Factorial, Sum, LinearSearch
الذيلي	1 (آخر عملية)	O(n)	O(1)*	TailFactorial, TailSum
الثنائي	2	O(n) - $O(2^n)$	O(log n) - O(n)	Fibonacci, MergeSort, BinarySum
المتعدد	k > 2	$O(k^n)$ - O(n!)	متغير	Permutations, N-Queens

*مع Tail Call Optimization

الأمثلة الكلاسيكية - يجب حفظها

1. Factorial (خطي)
Base Case: n = 0 or 1 → return 1
Recursive: n! = n × (n-1)!

2. Fibonacci (ثنائي)
Base Cases: F(0)=0, F(1)=1
Recursive: F(n) = F(n-1) + F(n-2)
التعقيد: $O(2^n)$ ⚠ غير فعال
الحل: Memoization أو DP

3. Merge Sort (ثنائي محسّن)
Divide: قسّم المصفوفة لنصفين
Conquer: فرز كل نصف عودياً
Combine: ادمج النصفين المرتبين
التعقيد: O(n log n) ✓
المساحة: O(n)

4. Towers of Hanoi (ثنائي)
Base Case: n=1 → انقل قرص واحد مباشرة
Recursive:
1. انقل (n-1) للعمود المساعد
2. انقل القرص الأكبر للهدف
3. انقل (n-1) من المساعد للهدف

عدد الحركات: $2^n - 1$

المشاكل الشائعة وحلولها

المشكلة	السبب	الحل
Stack Overflow	عمق كبير جداً	استخدم Tail Recursion أو حلقات
Infinite Recursion	لا يوجد/لا يُصل لـBase Case	تحقق من شروط الإيقاف
Overlapping Subproblems	حساب مكرر(Fibonacci)	**Memoization**أوDP
أداء ضعيف	نمو أسي	تحسين الخوارزمية أوPruning

- Tail Recursion نقطة حرجة

الشرط: الاستدعاء العودي هو **آخر عملية تماماً**

✗ ليست ذيلية:

```
return n + sum(n-1)  // عملية جمع بعد العودة
```

✓ذيلية:

```
return sum(n-1, accumulator + n)  // لا عمليات بعد العودة
```

4.تحليل التعقيد - قواعد سريعة

حساب عدد الاستدعاءات العودية

- **الخطي**: n استدعاء
- **الثنائي (Fibonacci)**: 2^(n+1) - 1
- **الثنائي (MergeSort)**: 2n - 1
- **المتعدد (Permutations)**: n!

عمق الاستدعاء(Stack Depth)

- **الخطي**: n
- **الثنائي المتوازن**: $\log_2(n)$
- **غير المتوازن**: n

---

5 .نقاط الامتحان الحرجة ⚠

أسئلة MCQ محتملة

1. **ما التعقيد الزمني لحذف عنصر من نهاية قائمة أحادية؟** → O(n)
2. **ما الفرق بين العودية الخطية والذيلية؟** →الذيلية لا تحتاج عمليات بعد العودة
3. **لماذا $O(2^n)$ Fibonacci ؟** → Overlapping Subproblems
4. **ما فائدة Sentinel Nodes ؟** →إلغاء الحالات الخاصة

أسئلة Trace محتملة

- تتبع Insertion Sort على مصفوفة صغيرة
- تتبع factorial(4) أو fibonacci(5)
- رسم شجرة الاستدعاءات
- حساب عدد الاستدعاءات الكلي

أسئلةTrue/False

- "القائمة المزدوجة تستخدم ذاكرة ضعف الأحادية" → True (للمؤشرات)
- "يمكن تحويلها لحلقة Tail Recursion" → True
- "المصفوفات توفر O(1) للإدراج" → False (إلا في النهاية)

---

6. صيغ يجب حفظها

- **Factorial**: $n! = n \times (n-1)!$، $0! = 1$
- **Fibonacci**: $F(n) = F(n-1) + F(n-2)$، $F(0)=0, F(1)=1$
- **Binary Search**: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$
- **Merge Sort**: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
- **Towers of Hanoi**: $2^n - 1$ حركة

نصيحة أخيرة :ركز على **التعقيدات الزمنية، الفروقات بين الأنواع، والأمثلة الكلاسيكية** -هذه تشكل 70% من الأسئلة المتوقعة!